Git Checkout & Stash

Workshop Handout — Basic Git Commands and Their Usage

1. What is Git and Why It Matters

Git is a distributed version control system. It tracks changes to files over time, lets multiple people work on the same codebase without overwriting each other's work, and allows you to move between different versions or lines of development safely.

This handout focuses on two commands that come up constantly in daily development work:

- git checkout — moving between branches, commits, and restoring files
- git stash — temporarily setting aside uncommitted work

2. One-Time Setup

Before using Git, configure your identity. This is done once per machine.

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --list
```

Starting a Repository

```
mkdir workshop-demo
cd workshop-demo
git init
git status
```

3. Basic Workflow (Add & Commit)

Checkout and stash only make sense once there is history and change to work with. Demonstrate the basic save cycle first:

```
echo "Hello Git" > file1.txt
git status                # untracked file
git add file1.txt         # stage it
git commit -m "Initial commit"
git log --oneline
```

4. git checkout — Usage and Explanation

git checkout is a multi-purpose command. It is used for three main things:

- Switching between existing branches
- Creating a new branch and switching to it in one step

- Restoring/discarding changes in files, or checking out an old commit

4.1 Switching Branches

Use this to move your working directory to point at a different branch's files.

```
git branch                # list branches
git branch feature-1      # create a branch
git checkout feature-1    # switch to it
git checkout main         # switch back
```

Why it matters: each branch can have its own independent changes. Checkout swaps the files in your working directory to match the branch you move to — nothing is lost, it is just "parked" on the branch it belongs to.

4.2 Create + Switch in One Step

```
git checkout -b feature-2
```

Equivalent to running git branch feature-2 followed by git checkout feature-2. This is the most commonly used form in real projects — used every time work begins on a new feature or fix.

4.3 Restoring Files

If you have made changes you no longer want, checkout can discard them and restore the last committed version.

```
echo "oops mistake" >> file1.txt
git status                    # shows modified file
git checkout -- file1.txt     # discard the change
cat file1.txt                 # back to last commit
```

You can also pull a specific file from another branch or commit without switching to it entirely:

```
git checkout main -- file1.txt
```

4.4 Checking Out a Commit (Detached HEAD)

```
git checkout <commit-hash>
```

This moves you to a specific point in history to inspect it. Git will warn about "detached HEAD" — this simply means you are not on a branch tip. Useful for viewing old code; not for making new commits directly.

4.5 A Note on Newer Commands

Recent Git versions split checkout's responsibilities into two clearer commands:

- git switch — for changing branches
- git restore — for restoring files

checkout still works everywhere and remains the command most tutorials and existing projects use, which is why it's worth teaching first.

5. git stash — Usage and Explanation

git stash temporarily saves uncommitted changes (both staged and unstaged) so you can switch context — change branches, pull updates, fix something urgent — without committing half-finished work, and without losing it.

5.1 The Core Problem It Solves

Imagine you are mid-way through editing a file, and an urgent bug needs fixing on another branch. You cannot checkout cleanly with uncommitted changes in the way (Git will often block the switch). Stash clears your working directory temporarily, keeping the changes safe on a stack.

5.2 Saving and Restoring Work

```
echo "half-done work" >> file1.txt
git status              # modified file
git stash                # save it away
git status              # working directory is clean
git stash list           # see saved stash
git stash pop            # bring changes back + remove from stash
```

git stash pop applies the most recent stash and removes it from the stash list. Use git stash apply instead if you want to apply it but keep a copy in the stash list.

5.3 Useful Variations

```
git stash save "message describing work"   # named stash
git stash -u                               # include untracked/new files
git stash list                             # multiple stashes stack up
git stash show -p stash@{0}                # preview a stash's contents
git stash apply stash@{1}                  # apply a specific, older stash
git stash drop stash@{0}                   # delete one specific stash
git stash clear                            # delete all stashes
```

Stashes stack up like a pile of plates — stash@{0} is always the most recent. Naming them with a message becomes important once more than one is on the stack.

6. Combined Real-World Workflow

Checkout and stash are most powerful together. This sequence mirrors a very common interruption scenario and is a strong closing demo for a workshop:

```
git checkout -b bugfix        # 1. start new branch
echo "code" >> file1.txt      # 2. make changes
git stash                     # 3. urgent switch needed, stash it
git checkout main             # 4. go fix something on main
echo "hotfix" >> file1.txt
git commit -am "hotfix"
git checkout bugfix           # 5. come back to bugfix
git stash pop                 # 6. resume exactly where you left off
```

7. Quick Reference Cheat Sheet

Command	Usage

`git checkout <branch>`	Switch to an existing branch
`git checkout -b <branch>`	Create a new branch and switch to it
`git checkout -- <file>`	Discard changes in a file
`git checkout <branch> -- <file>`	Restore a file from another branch
`git checkout <commit>`	View an old commit (detached HEAD)
`git stash`	Save uncommitted changes, clean working directory
`git stash -u`	Also stash untracked/new files
`git stash list`	Show all saved stashes
`git stash pop`	Apply latest stash and remove it from the list
`git stash apply`	Apply latest stash, keep it in the list
`git stash drop stash@{n}`	Delete one specific stash
`git stash clear`	Delete all stashes

Tech Surya IT Solution Ameya Hattarke